Ch. 4 Notes

Lists, Stacks, Queues, Deques

List ADT operations

`size()`, `empty()`, `front()`, `back()`, `append(x)`, `prepend(x)`, `insertAfter(cur, x)`, `remove(cur)`, `search(x)`, traversal.

SLL node

```
struct Node {
    int value;
    Node* next;        // nullptr = end
};
struct SLL { Node* head = nullptr;
             Node* tail = nullptr;
             size_t sz  = 0; };
```

DLL node

```
struct Node {
    int value;
    Node* next;
    Node* prev;
};
```

Invariant: for all nodes except head/tail, `n->next->prev == n` and `n->prev->next == n`.

Append + prepend (SLL, with tail)

Append: new node, `tail->next = new`, `tail = new`. If empty, also set head.
Prepend: new node, `new->next = head`, `head = new`. If empty, also set `tail`.
Cost: both $O(1)$.

Insert-after, remove (SLL)

insertAfter(cur, x): three cases (empty, append, middle). Always $O(1)$ once you have `cur`.
remove needs the *predecessor* pointer – if you only have `cur`, you walk from head to find prev $\rightarrow O(n)$.
This is why DLL exists.

DLL advantages over SLL

- `remove(cur)` in $O(1)$ when you have a pointer.
- `insertBefore(cur, x)` is as cheap as `insertAfter`.
- Reverse traversal is a natural `for (n=tail; n; n=n->prev)`.

Cost: two pointers per node (vs. SLL's one).

Sentinel (dummy) nodes

Fixed head *and* fixed tail node that hold no data. Every real node is between them. Collapses the three cases (empty / first / last / middle) into one uniform "splice between prev and next" code path. `std::list` uses a circular sentinel.

Linked-list search

Linear: walk from head until match or `nullptr`. Cost: $O(n)$. Cache-hostile: non-contiguous pointer chasing.

Stack ADT

LIFO. `push(x)`, `pop()`, `top()`, `empty()`. Implement with SLL (push/pop at head) or array/vector (push/pop at back). Both $O(1)$. `std::stack` → defaults to `std::deque`.

Queue ADT

FIFO. `enqueue(x)`, `dequeue()`, `front()`. SLL with both head and tail: enqueue at tail, dequeue at head. Array alone is bad (dequeue from front is $O(n)$); use a ring buffer or `std::deque`. `std::queue` → `std::deque`.

Deque ADT

Push/pop at both ends. DLL is natural. Or block-based dynamic array (`std::deque`).

Decision matrix

Op	arr	SLL	DLL	sDLL
`v[i]`	1	n	n	n
append*	1	1	1	1
prepend	n	1	1	1
insertAfter	n	1	1	1
insertBefore	n	n	1	1
remove (have ptr)	n	n	1	1
reverse traverse	n	n	n	n
bytes/elem	lo	+ 1 ptr	+ 2 ptrs	+ 2 ptrs
cache-friendly	yes	no	no	no

*Array append is amortized $O(1)$ via doubling — single operations can be $O(n)$ on the resize step.

When to pick each

- **Array / vector.** Default. Random access. Dense memory.
- **SLL.** Rarely the right answer. Pick only if memory per element matters and all ops are at head.
- **DLL.** Mid-list inserts/removes with a held pointer, bidirectional traversal.
- **Sentinel DLL.** Same as DLL, with cleaner code. What `std::list` is.

Top gotchas

- Dereferencing `cur->next` after deletion (use-after-free).
- Forgetting to update `tail` on head operations.
- Missing the empty-list special case in `remove`.
- Assuming linked lists beat vectors on middle-insert (usually false under 10^4).
- On a DLL, updating `next` without the twin `prev` update.